

Laboratorio de SonarQube — Soporte360

Parte 1 — Recorrido funcional

¿La aplicación parece funcional?

Sí. Pudimos iniciar sesión, revisar el dashboard, buscar y filtrar tickets, cambiar estados y responsables, responder mensajes, crear un ticket, exportar la lista en CSV y consultar clientes.

¿Encontraste algún problema evidente solamente usando la interfaz?

No. Durante el recorrido básico, las funciones principales respondieron de la manera esperada.

¿Podrías afirmar, basándote solamente en la UI, que el código es seguro y mantenible? ¿Por qué?

No. La interfaz permite comprobar que la aplicación funciona, pero no muestra problemas internos como credenciales expuestas, código inseguro, funciones complejas o falta de pruebas. Para detectar esos problemas fue necesario analizar el código con SonarQube.

Parte 2 — Pruebas y cobertura

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	1.08	14.28	10	1.08	
src	0	0	0	0	
App.tsx	0	0	0	0	1-60
types.ts	0	0	0	0	1-42
src/components	0	0	0	0	
CustomerList.tsx	0	0	0	0	1-17
Dashboard.tsx	0	0	0	0	1-48
LoginScreen.tsx	0	0	0	0	1-36
NewTicketModal.tsx	0	0	0	0	1-30
Sidebar.tsx	0	0	0	0	1-20
TicketDetail.tsx	0	0	0	0	1-41
TicketList.tsx	0	0	0	0	1-25
Topbar.tsx	0	0	0	0	1-8
src/data	0	0	0	0	
seed.ts	0	0	0	0	1-67
src/services	0	0	0	0	
authService.ts	0	0	0	0	1-35
exportService.ts	0	0	0	0	1-25
storageService.ts	0	0	0	0	1-30
ticketService.ts	0	0	0	0	1-86
src/utils	9.09	50	40	9.09	
dashboard.ts	0	0	0	0	1-44
format.ts	41.17	75	66.66	41.17	2-11
search.ts	0	0	0	0	1-16

¿La existencia de algunos tests implica necesariamente buena cobertura?

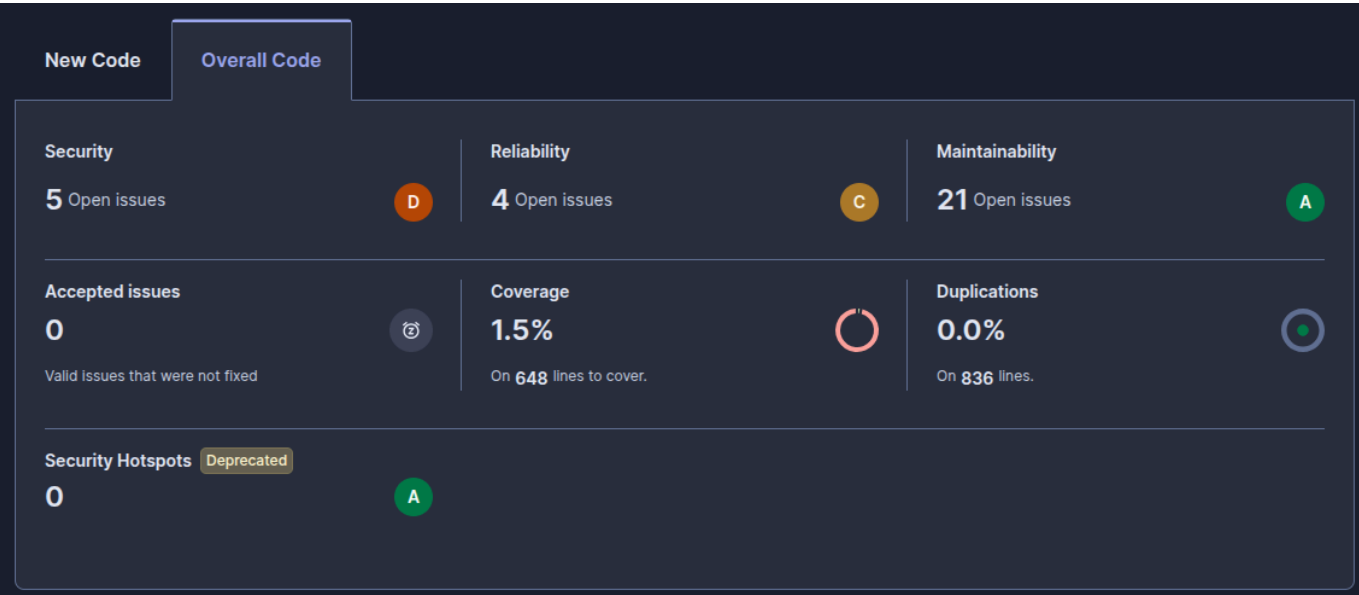
No. Aunque los tests pasen, pueden estar evaluando solo una parte pequeña del sistema. En este caso, los dos tests cubren únicamente funciones simples de formato y la cobertura total es de aproximadamente 1 %.

¿Qué partes importantes de la aplicación no parecen estar cubiertas?

No están cubiertas algunas de las funciones principales de la aplicación, como por ej. la autenticación, la creación y modificación de tickets o las funciones de búsqueda y filtros, entre otros. Por eso, los tests actuales no permiten asegurar que los flujos importantes funcionen correctamente.

Actividad de análisis

A. Panorama general inicial



Métrica	Resultado inicial
Security	5 issues — D
Reliability	4 issues — C
Maintainability	21 issues — A
Security Hotspots	0 — A
Coverage	1,5 % sobre 648 líneas
Duplicated Lines	0,0 %
Cyclomatic Complexity	201
Cognitive Complexity	143

Un mismo issue puede afectar más de una categoría. Por ejemplo, un problema de accesibilidad puede impactar en Reliability y Maintainability, por lo que los totales no deben sumarse entre sí.

B. Hallazgos concretos

Archivo	Hallazgo	Categoría	Severidad	¿Por qué importa?
authService.ts:4	Contraseña de demostración escrita en el frontend	Security	Media	Cualquier persona que reciba la aplicación puede verla.
authService.ts:34	Token generado con Math.random()	Security	Media	No es una fuente segura para generar identificadores de sesión.
exportService.ts:23-24	Ejecución dinámica mediante new Function()	Security	Alta	Podría ejecutar código no autorizado si la expresión fuera controlada por un usuario.
ticketService.ts:3	Endpoint definido con HTTP	Security	Baja	Una conexión futura podría enviar información sin cifrar.
ticketService.ts:23-66	calculateSlaRisk() tenía complejidad cognitiva 68	Maintainability	Alta	Era difícil modificar sus reglas sin introducir errores.
App.tsx:46-47	Ternarios anidados para los encabezados	Maintainability	Media	La navegación era difícil de leer y ampliar.
search.ts:9-11	Excepción capturada y descartada	Maintainability	Baja	Los errores de búsqueda quedaban ocultos.
NewTicketModal.tsx:21	Un div no interactivo cerraba el modal	Reliability / Maintainability	Media	El control no era adecuado para teclado o tecnologías de asistencia.
LoginScreen.tsx:31 y NewTicketModal.tsx:28	Botones sin type explícito	Reliability	Media	Podían provocar envíos accidentales si cambiaba el formulario.
dashboard.ts:4-43	Dos funciones repetían el cálculo de estadísticas	Lógica repetida	Media	Las estadísticas podían quedar inconsistentes si se actualizaba una sola función.
format.test.ts	Los tests cubrían solamente dos helpers	Coverage	Alta	Los flujos principales no estaban protegidos contra regresiones.

Evidencia de los hallazgos

Ejecución dinámica con new Function():

Soporte360 - Laboratorio SonarQube / Issues / Make sure that this dynamic injection or execution ...

Issues

26 issues

src/services/exportService.ts

Make sure that this dynamic injection or execution of code is safe.

Make sure that this dynamic injection or execution of code is safe.

src/services/authService.ts

Review this potentially hard-coded password.

Make sure that using this pseudorandom number generator is safe here.

src/.../LoginScreen.tsx

Add an explicit "type" attribute

Make sure that this dynamic injection or execution of code is safe.

Dynamic code execution should not use user-controlled data [typescript:S1523](#)

Line affected: L23 • Effort: 30min • Introduced: 1 hour ago

Open Not assigned cwe ...

Where is the issue?

Why is this an issue?

How can I fix it?

Activity

More info

18

anchor.click();

19

URL.revokeObjectURL(url);

20

}

21

22

export function evaluateCustomScore(expression: string, openTickets: number) {

23

const fn = new Function('openTickets', 'return \$(expression)');

24

return Number(fn(openTickets));

25

}

26

Make sure that this dynamic injection or execution of code is safe.

Make sure that this dynamic injection or execution of code is safe.

Software qualities impacted

Security High

Maintainability Low

Code attribute

Responsibility | Not trustworthy

Problemas de seguridad en authService.ts:

Soporte360 - Laboratorio SonarQube / Issues / Review this potentially hard-coded password.

Issues

26 issues

src/services/exportService.ts

Make sure that this dynamic injection or execution of code is safe.

Make sure that this dynamic injection or execution of code is safe.

src/services/authService.ts

Review this potentially hard-coded password.

Make sure that using this pseudorandom number generator is safe here.

src/.../LoginScreen.tsx

Add an explicit "type" attribute to this button.

Review this potentially hard-coded password.

Credentials should not be hard-coded [typescript:S2068](#)

Line affected: L4 • Effort: 30min • Introduced: 1 hour ago

Open Not assigned cwe ...

Where is the issue?

Why is this an issue?

How can I fix it?

Activity

More info

3

4

5

6

7

8

9

10

11

12

13

Review this potentially hard-coded password.

Show next few lines of code

Show all lines

Software qualities impacted

Security Medium

Code attribute

Responsibility | Not trustworthy

Complejidad de calculateSlaRisk():

Soporte360 - Laboratorio SonarQube / Issues / Refactor this function to reduce its Cognitive Com...

Issues

src/.../Dashboard.tsx

Mark the props of the component as read-only.

src/.../LoginScreen.tsx

Mark the props of the component as read-only.

src/.../NewTicketModal.tsx

Mark the props of the component as read-only.

src/.../Sidebar.tsx

Mark the props of the component as read-only.

Refactor this function to reduce its Cognitive Complexity from 68 to the 15 allowed.

Cognitive Complexity of functions should not be too high [typescript:S3776](#)

Line affected: L23 • Effort: 58min • Introduced: 1 hour ago

Open Not assigned brain-overload

Where is the issue? Why is this an issue? How can I fix it? Activity More info

Soporte360 - Laboratorio SonarQube > src/services/ticketService.ts

Open in IDE See all issues in this file

Show previous few lines of code Show all lines

18 || updatedAt: new Date().toISOString(),

19 || comments: []

20 || };

21 || }

22 ||

23 || export function calculateRisk(ticket: Ticket, now = new Date()): number {

24 ||

25 || let score = 0;

26 || const ageHours = (now.getTime() - new Date(ticket.createdAt).getTime()) / 3600000;

27 || if (ticket.status !== 'Resuelto') {

Refactor this function to reduce its Cognitive Complexity from 68 to the 15 allowed.

Software qualities impacted

Maintainability High

Code attribute

Adaptability Not focused

Problema de accesibilidad del modal:

Soporte360 - Laboratorio SonarQube / Issues / Add an explicit "type" attribute to this button.

Issues

src/services/authService.ts

Review this potentially hard-coded password.

Make sure that using this pseudorandom number generator is safe here.

src/.../LoginScreen.tsx

Add an explicit "type" attribute to this button.

src/.../NewTicketModal.tsx

Avoid non-native interactive elements. If using native HTML is not possible, add an appropriate role and support for tabbing, mouse, keyboard, and touch inputs to an interactive content element.

Add an explicit "type" attribute

Add an explicit "type" attribute to this button.

"<button>" elements should have a valid "type" attribute, explicitly set when associated with a " <form>" [typescript:S9011](#)

Line affected: L28 • Effort: 2min • Introduced: 1 hour ago

Open Not assigned vue

Where is the issue? Why is this an issue? Activity More info

26 ||

27 ||

28 || <div className="modal-actions"><button type="button" className="secondary" onClick={onClose}>Cancelar</button>

29 || <button className="primary">Crear ticket</button></div>

30 ||

31 ||

Add an explicit "type" attribute.

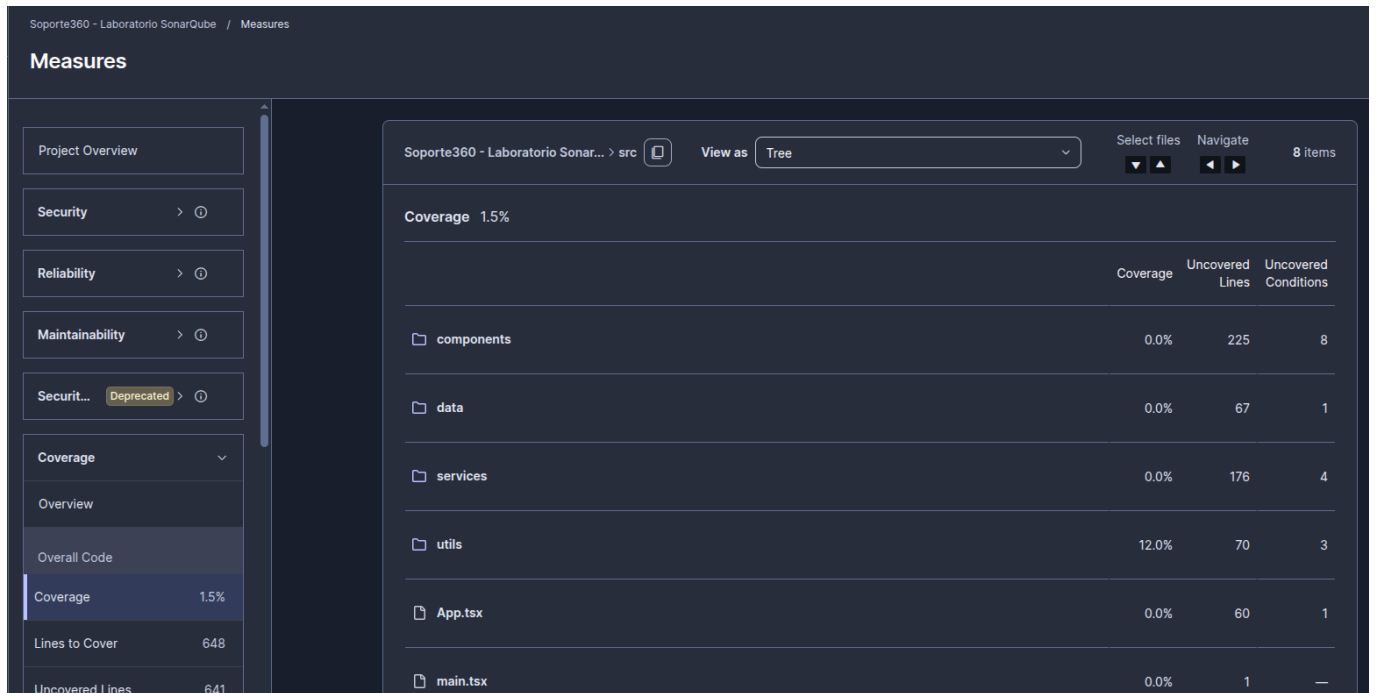
Software qualities impacted

Reliability Medium

Code attribute

Intentionality Not logical

Cobertura inicial por archivo:



C. Relación entre los hallazgos y el software

Hallazgo	Funcionalidad afectada	Posible efecto visible
Contraseña expuesta	Inicio de sesión	Una credencial válida podría ser conocida o reutilizada.
Token generado con Math.random()	Sesión	Un token predecible podría facilitar la suplantación de un usuario.
Uso de new Function()	Exportación o futuras funciones	Podría ejecutarse JavaScript no autorizado si se reutilizara con datos del usuario.
Endpoint HTTP	Auditoría de tickets	Los datos podrían viajar sin cifrado si se implementara la petición real.
Complejidad del SLA	Dashboard y detalle del ticket	Un cambio en las reglas podría calcular mal la prioridad de atención.
Ternarios anidados	Navegación	Al agregar vistas podría mostrarse un encabezado incorrecto.
Errores ignorados en búsqueda	Filtros y búsqueda	Una consulta inválida podía cambiar silenciosamente los resultados.
Modal no accesible	Creación de tickets	Un usuario de teclado podía tener dificultades para cerrarlo.
Botones sin tipo	Login y creación de tickets	Un cambio futuro podía generar envíos involuntarios.
Estadísticas repetidas	Dashboard	Dos secciones podían mostrar números diferentes.
Cobertura baja	Aplicación completa	Una regresión en un flujo importante podía pasar sin ser detectada.

D. Refactorización

Se corrigieron ocho grupos de hallazgos:

1. Se eliminó evaluateCustomScore() y, con ella, el uso de new Function().
2. Se reemplazó Math.random() por crypto.randomUUID() y se cambió el endpoint de auditoría a HTTPS.

3. Se dividió el cálculo SLA en reglas y funciones auxiliares.
4. Se reemplazaron los ternarios de App.tsx por una configuración por vista.
5. Se agregó type="submit" a los botones de los formularios.
6. Se cambió la búsqueda con expresiones regulares por una búsqueda literal.
7. Se eliminó la función duplicada de estadísticas.
8. Se agregaron diez pruebas para SLA, búsqueda, filtros y dashboard.

Los cambios principales quedaron así:

```
const LEGACY_AUDIT_ENDPOINT = 'https://legacy-audit.internal.local/events';

export function calculateSlaRisk(ticket: Ticket, now = new Date()): number {
  if (ticket.status === 'Resuelto') return 0;

  const ageHours = (now.getTime() - new Date(ticket.createdAt).getTime()) / 3600000;
  const categoryBonus = ticket.priority === 'Urgente'
    ? (URGENT_CATEGORY_BONUS[ticket.category] ?? 3)
    : 0;
  const commonBonus = (ticket.title.length > 80 ? 2 : 0)
    + (ticket.comments.length === 0 && ageHours > 6 ? 7 : 0)
    + (ticket.comments.length > 8 ? 3 : 0);
  const score = BASE_RISK[ticket.priority]
    + sumAgeBonuses(ticket.priority, ageHours)
    + (ticket.assignee ? 0 : UNASSIGNED_BONUS[ticket.priority])
    + statusBonus(ticket)
    + categoryBonus
    + commonBonus;

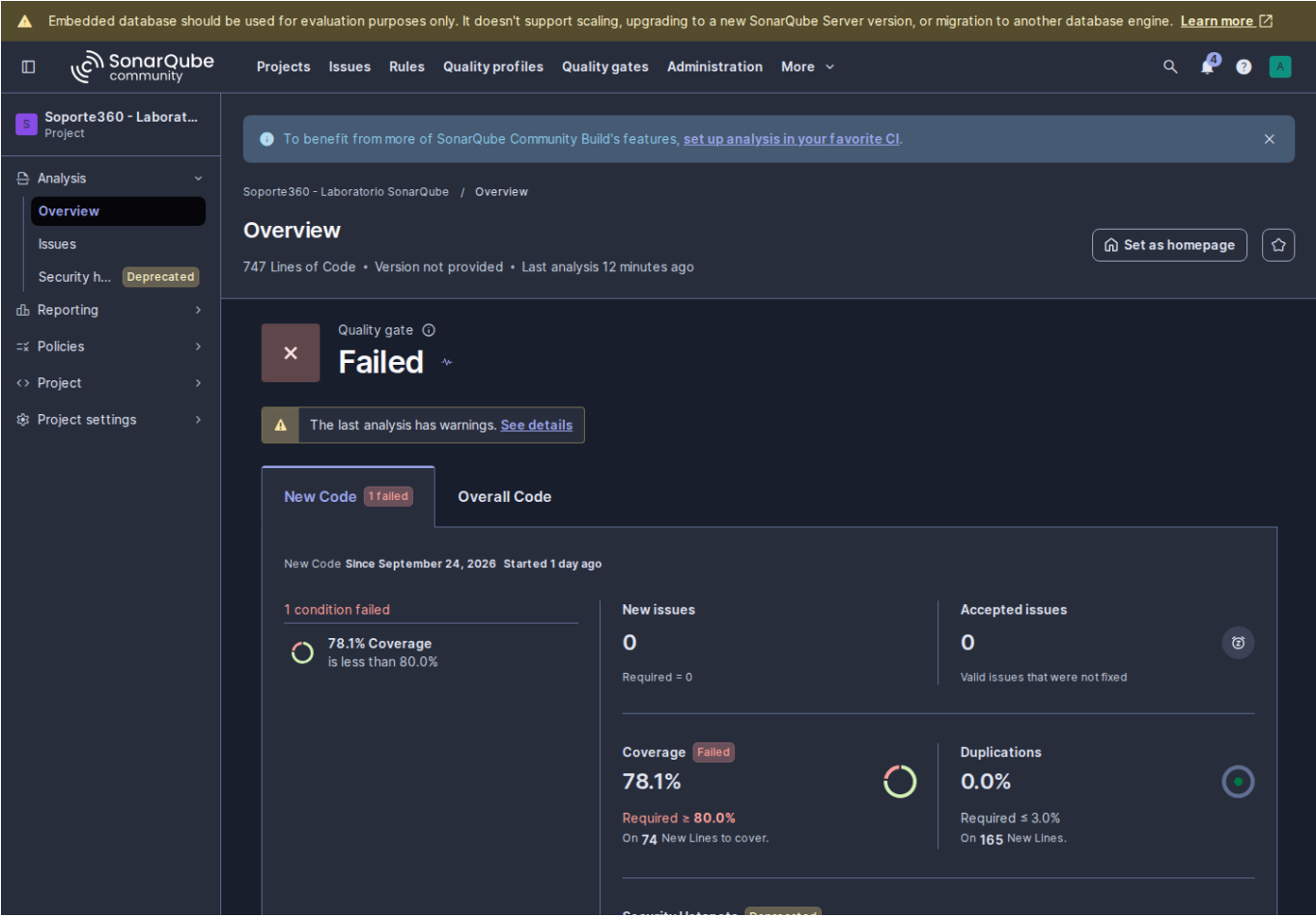
  return Math.min(100, Math.max(0, Math.round(score)));
}
```

```
const normalizedQuery = query.trim().toLowerCase();
if (normalizedQuery) {
  result = result.filter((ticket) => [
    ticket.title,
    ticket.customerName,
    ticket.customerEmail
  ].some((value) => value.toLowerCase().includes(normalizedQuery)));
}
```

Verificación local

Verificación	Antes	Después
Tests aprobados	2	12
Archivos de pruebas	1	2
Statements	1,08 %	15,79 %
Branches	14,28 %	68,25 %
Functions	10 %	26,92 %
Build de producción	Correcto	Correcto

Comparación en SonarQube



Métrica	Antes	Después
Security	5 issues — D	1 issue — C
Reliability	4 issues — C	2 issues — A
Maintainability	21 issues — A	15 issues — A
Security Hotspots	0 — A	0 — A
Coverage	1,5 %	20,5 %
Line Coverage	1,1 %	15,8 %
Branch Coverage	14,3 %	68,3 %
Duplicated Lines	0,0 %	0,0 %
Cyclomatic Complexity	201	176
Cognitive Complexity	143	70

SonarQube confirmó que se resolvieron los issues relacionados con new Function(), Math.random(), HTTP, la complejidad de calculateSlaRisk(), los ternarios de App.tsx, la excepción ignorada y los botones sin tipo.

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQube
community

ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

Soporte360 - Laborat...
Project

Analysis

- Overview
- Issues
- Security h... Deprecated

ReportingPoliciesProjectProject settings

Soporte360 - Laboratorio SonarQube / Issues

Issues

My IssuesAll

Filters

Issues in new code

Software Quality

- Security1
- Reliability2
- Maintainability15

Severity

- Blocker0
- High0
- Medium7
- Low11
- Info0

Code attribute

Scope

Status

Compliance Category

☐ Bulk Change

16 issues1h 45min effort

Sort byPriority

src/services/authService.ts

☐ Review this potentially hard-coded password.

Responsibility

SecurityMedium

cwe secret

OpenNot assigned

L4 • 30min effort • 1 day ago

src/components/NewTicketModal.tsx

☐ Avoid non-native interactive elements. If using native HTML is not possible, add an appropriate role and support for tabbing, mouse, keyboard, and touch inputs to an interactive content element.

Consistency

ReliabilityMediumMaintainabilityLow

accessibility react

OpenNot assigned

L21 • 5min effort • 1 day ago

src/components/TicketDetail.tsx

☐ Extract this nested ternary operation into an independent statement.

Intentionality

MaintainabilityMedium

editable-source confusing

OpenNot assigned

L21 • 5min effort • 1 day ago

La cobertura mejoró, aunque varios componentes visuales todavía permanecen en 0 %:

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQube
community

ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

Soporte360 - Laborat...
Project

AnalysisReporting

- Measures
- Activity
- Quality gate histo... New

PoliciesProjectProject settings

Soporte360 - Laboratorio SonarQube / Measures

Measures

Project Overview

SecurityReliabilityMaintainabilitySecurity ... Deprecated

Coverage

- Overview
- New Code
- Coverage78.1%
- Lines to Cover74
- Uncovered Lines21
- Line Coverage71.6%
- Conditions to Cover31

Soporte360 - Laboratorio Sonar...View asList

Select filesNavigate

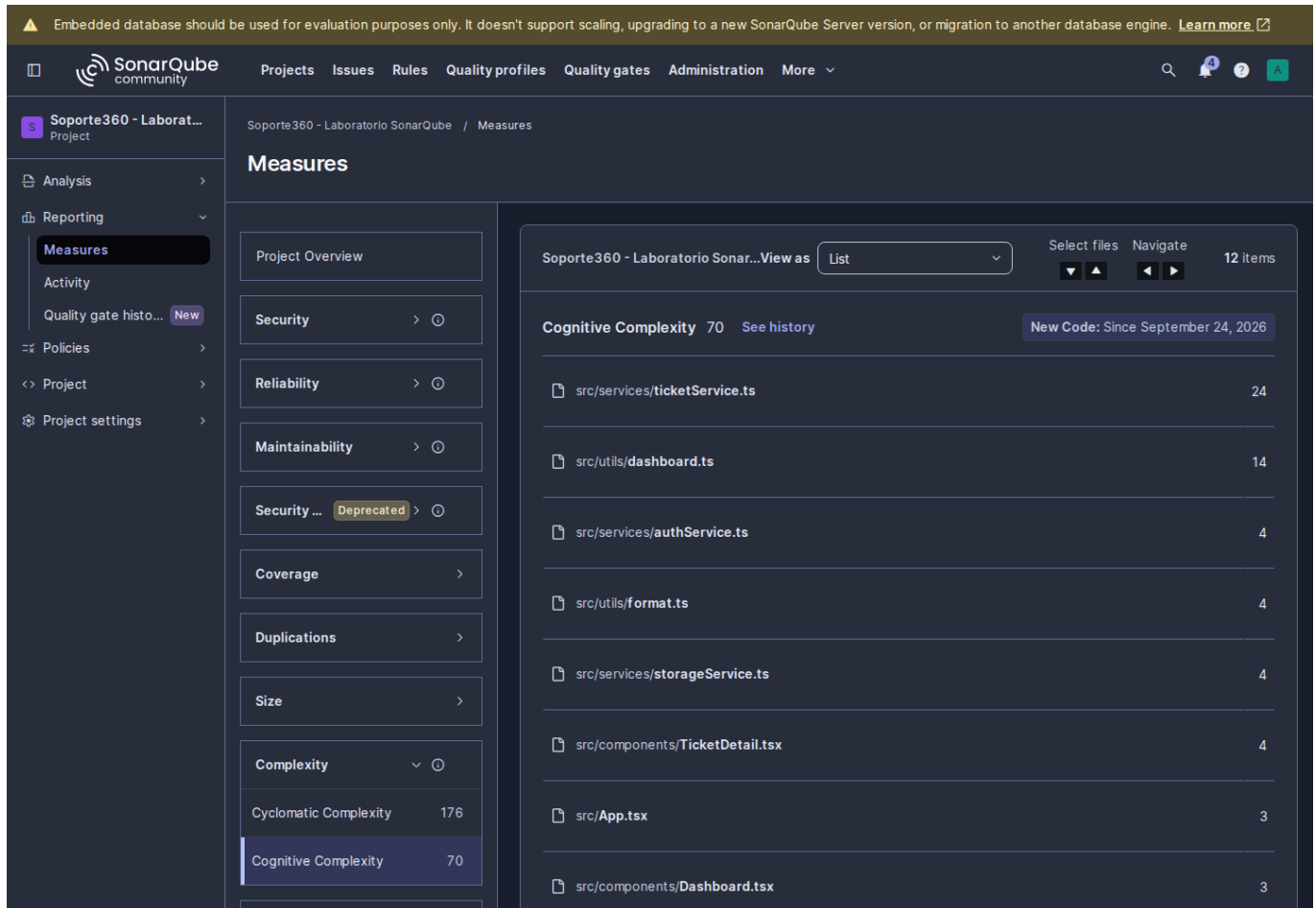
19 items

Coverage20.5%See history

New Code: Since September 24, 2026

	Coverage	Uncovered Lines	Uncovered Conditions
src/App.tsx	0.0%	74	1
src/services/authService.ts	0.0%	35	1
src/components/CustomerList.tsx	0.0%	17	1
src/components/Dashboard.tsx	0.0%	48	1
src/services/exportService.ts	0.0%	20	1
src/components/LoginScreen.tsx	0.0%	36	1
src/main.tsx	0.0%	1	—

La complejidad cognitiva total bajó de 143 a 70:



Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

Soporte360 - Laboratorio SonarQube / Measures

Measures

- Project Overview
- Security
- Reliability
- Maintainability
- Security ... Deprecated
- Coverage
- Duplications
- Size
- Complexity
 - Cyclomatic Complexity 176
 - Cognitive Complexity 70

Soporte360 - Laboratorio Sonar... View as List Select files Navigate 12 items

Cognitive Complexity 70 See history New Code: Since September 24, 2026

src/services/ticketService.ts	24
src/utills/dashboard.ts	14
src/services/authService.ts	4
src/utills/format.ts	4
src/services/storageService.ts	4
src/components/TicketDetail.tsx	4
src/App.tsx	3
src/components/Dashboard.tsx	3

Quedaron pendientes la contraseña de demostración, la accesibilidad del modal, los problemas de contraste y algunos code smells menores. El Quality Gate falló porque la cobertura del código nuevo todavía no alcanza el 80 %.

E. Conclusión

1. ¿Qué problemas encontró SonarQube que no eran evidentes al usar la aplicación?

Encontró una credencial en el frontend, generación insegura de tokens, ejecución dinámica de código, uso de HTTP, complejidad elevada, problemas de accesibilidad y una cobertura muy baja. La aplicación funcionaba, pero estos riesgos no podían verse desde la interfaz.

2. ¿Qué diferencia existe entre que el software funcione y que tenga buena calidad interna?

Que funcione significa que cumple sus tareas en los casos probados. La calidad interna también requiere que sea seguro, fácil de entender y modificar, accesible y respaldado por pruebas. Un sistema puede funcionar hoy y ser difícil o riesgoso de mantener mañana.

3. ¿Qué hallazgo fue el más riesgoso y por qué?

El uso de `new Function()`, porque podía ejecutar texto como JavaScript. Si esa función llegaba a recibir datos controlados por un usuario, podía permitir la ejecución de código no autorizado.

4. ¿Qué métrica cambió más después de la refactorización?

La complejidad cognitiva tuvo la mayor reducción absoluta: bajó de 143 a 70. En las métricas de pruebas, la cobertura de ramas fue la que más mejoró: pasó de 14,3 % a 68,3 %.

5. ¿Qué problemas no se solucionan simplemente aumentando la cobertura?

La cobertura no corrige credenciales expuestas, código inseguro, protocolos sin cifrar, problemas de accesibilidad, duplicación o complejidad. Las pruebas ayudan a detectar regresiones, pero esos problemas necesitan cambios en el código o en el diseño.

6. ¿Tendría sentido usar un Quality Gate? ¿Qué condiciones exigirías?

Sí. Para el código nuevo exigiríamos:

- ninguna vulnerabilidad alta;
- calificación A en Security y Reliability;
- todos los Security Hotspots revisados;
- cobertura mínima del 80 %;
- duplicación menor al 3 %;
- ninguna función por encima del límite de complejidad cognitiva.

En este análisis el Quality Gate fue útil porque mostró que, aunque la aplicación mejoró, todavía falta aumentar la cobertura de las partes nuevas.